In [ ]:
```
import torch
import pickle
import io
import numpy as np
import pandas as pd
from sklearn.svm import SVR
from sklearn.linear_model import Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

class CPU_Unpickler(pickle.Unpickler):
    def find_class(self, module, name):
        if module == 'torch.storage' and name == '_load_from_bytes':
            return lambda b: torch.load(io.BytesIO(b), map_location='cpu')
        return super().find_class(module, name)

def prep(tensor):
    arr = tensor.numpy()
    return np.mean(arr, axis=1) if len(arr.shape) == 3 else arr

with open('data_cmu_mosei.pkl', 'rb') as f:
    data = CPU_Unpickler(f).load()

X_sets = {
    "Text": (prep(data['text_train']), prep(data['text_test'])),
    "Audio": (prep(data['audio_train']), prep(data['audio_test'])),
    "Visual": (prep(data['visual_train']), prep(data['visual_test']))
}

y_train, y_test = prep(data['label_train']).flatten(), prep(data['label_test']).flatten()


scalers = {k: StandardScaler() for k in X_sets}
X_scaled = {k: (scalers[k].fit_transform(X_sets[k][0]), scalers[k].transform(X_sets[k][1])) for k in X_sets}

results = []
preds_dict = {}

for name, (xtr, xte) in X_scaled.items():
    model = SVR().fit(xtr, y_train)
    preds = model.predict(xte)
    preds_dict[name] = preds
    results.append({"Model": f"{name}-Only", "MSE": mean_squared_error(y_test, preds), "R2": r2_score(y_test, preds)})

X_early_tr = np.concatenate([X_scaled[k][0] for k in X_scaled], axis=1)
X_early_te = np.concatenate([X_scaled[k][1] for k in X_scaled], axis=1)
model_early = SVR().fit(X_early_tr, y_train)
preds_early = model_early.predict(X_early_te)
results.append({"Model": "Early Fusion", "MSE": mean_squared_error(y_test, preds_early), "R2": r2_score(y_test, preds_early)})
preds_dict["Early"] = preds_early

meta_tr = np.column_stack([SVR().fit(X_scaled[k][0], y_train).predict(X_scaled[k][0]) for k in X_scaled])
meta_te = np.column_stack([SVR().fit(X_scaled[k][0], y_train).predict(X_scaled[k][1]) for k in X_scaled])
meta_model = Ridge().fit(meta_tr, y_train)
preds_late = meta_model.predict(meta_te)
results.append({"Model": "Late Fusion", "MSE": mean_squared_error(y_test, preds_late), "R2": r2_score(y_test, preds_late)})
preds_dict["Late"] = preds_late

results_df = pd.DataFrame(results)
print(results_df.to_string())


h3_df = pd.DataFrame({'Actual': y_test, 'Text_Pred': preds_dict['Text'], 'Late_Pred': preds_dict['Late']})
missed = h3_df[(h3_df['Text_Pred'].abs() < 0.2) & (h3_df['Actual'].abs() > 1.0)]
```

```
C:\Users\anjel\AppData\Local\Temp\ipykernel_4552\1727400087.py:15: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  return lambda b: torch.load(io.BytesIO(b), map_location='cpu')
         Model       MSE        R2
0     Text-Only  0.461719  0.620368
1    Audio-Only  1.102670  0.093368
2   Visual-Only  1.144711  0.058801
3  Early Fusion  0.460197  0.621619
4   Late Fusion  0.455535  0.625452

--- Hypothesis 3 (Text vs Late Fusion) ---
Improvement: 1.20%
```

In [ ]:
```
mask = (np.abs(preds_dict['Text']) < 0.2) & (np.abs(y_test) > 1.0)
y_h3 = y_test[mask]
```

```
h3_data = {
    "Model": ["Text-Only", "Early Fusion", "Late Fusion"],
    "MSE": [
        mean_squared_error(y_h3, preds_dict['Text'][mask]),
        mean_squared_error(y_h3, preds_dict['Early'][mask]),
        mean_squared_error(y_h3, preds_dict['Late'][mask])
    ]
}

h3_df = pd.DataFrame(h3_data)

print("\n" + "="*45)
print("--- Hypothesis 3 Analysis (Filtered Samples) ---")
print(f"Number of ambiguous samples found: {len(y_h3)}")
print("-" * 45)
print(h3_df.to_string(index=False))
print("="*45 + "\n")

base_mse = h3_df.loc[0, 'MSE']
h3_df['Improvement (%)'] = ((base_mse - h3_df['MSE']) / base_mse) * 100
print("Improvement over Text-Only Baseline:")
print(h3_df[['Model', 'Improvement (%)']].to_string(index=False))
```

```
=============================================
--- Hypothesis 3 Analysis (Filtered Samples) ---
Number of ambiguous samples found: 94
---------------------------------------------
       Model      MSE
   Text-Only 2.713632
Early Fusion 2.705015
 Late Fusion 2.681130
=============================================

Improvement over Text-Only Baseline:
       Model  Improvement (%)
   Text-Only         0.000000
Early Fusion         0.317557
 Late Fusion         1.197743
```

In [ ]: